



UNIVERSIDADE
DE ÉVORA

Assignment 2 – Termination Detection
Algorithm for a Distributed Basic Algorithm

Programação Paralela

Done By:

Tiago Pinto m68106

João Lóios m68290

Mestrado em Engenharia Informática

Objective

With this work, we aim to implement termination detection algorithms for a basic distributed algorithm using the MPI communication environment. The main objective is to study and understand how it is possible to determine when a distributed computation has terminated, considering that processes may be active, passive, or still have messages in transit.

For this work, we selected two distributed termination detection algorithms: the Dijkstra–Scholten algorithm and Safra’s algorithm. Both algorithms allow the detection of global termination through the exchange of control messages between processes, but they use different approaches. The Dijkstra–Scholten algorithm is based on the creation of a dynamic dependency tree between processes, while Safra’s algorithm uses a token that circulates between processes to verify whether the entire computation has finished.

The implementation of these algorithms allows us to analyze their behaviour, compare their termination detection strategies, and understand the challenges associated with synchronization and communication in distributed systems.

Description of the Implemented Algorithms

Dijkstra-Scholten

Dijkstra–Scholten's algorithm is a distributed termination detection algorithm that determines when a distributed computation has finished, meaning that all processes are passive and there are no messages still being processed or in transit. The algorithm works by creating a dependency tree between processes, where each process has a parent and possibly several children. When a process sends a message to another process, it creates a dependency relation and the receiver becomes a child of the sender.

A process can only report termination when it becomes passive and all of its children have also finished. When these conditions are met, the process sends an acknowledgement message to its parent, removing the dependency. This acknowledgement propagation continues through the tree until it reaches the initiator. When the initiator becomes passive and has no remaining children, it means that every process has terminated and a termination message is broadcast to all processes.

The main challenge of the algorithm is maintaining the correct dependency tree while messages are still being exchanged. A process may receive messages from multiple processes, but only the first sender becomes its parent. Messages received from other processes are immediately acknowledged, preventing the creation of invalid dependencies. This guarantees that the tree structure remains correct and that termination is only detected when the whole distributed system has finished.

Safra

Safra's algorithm is a distributed termination algorithm that detects termination when, in a distributed system, all processes are passive and there are no messages still in transit between processes. The algorithm operates by assigning a numeric value and a color to each process. The numeric value represents the traffic of the messages in the system, by adding 1 to the value if the process sends a message and decreasing 1 if it receives one, the system can later check the overall balance to see if any message is still in transit. This balance is calculated through a token going process after process adding the total balance, this token starts in a process, and when this becomes passive goes to the next, this repeats until the token gives a full circle and the messages balance is 0. When this token reaches the begging and the algorithm does not finish, all of the tokens values are restored before starting a new round.

This strategy is flawless by itself, if not for the fact that a process can send a message to another that the token already visited. To account for this, the algorithm assigns a color to each process and the token, if nothing happens the color remains white. if the process send a message to a process behind it, in the order assigned in the algorithm, this token becomes black, and when the token passes, the token becomes black, resenting the color of the preciously black process. If this black token completes a full circle, even if the message count is 0, the algorithm does not finish and another round is started.

Analysis of the algorithms

Dijkstra-Scholten

First, we defined two types of control messages, an ACK_MESSAGE used to notify the parent process that a child has finished, and a TERMINATE_MESSAGE used to signal that the entire distributed computation has terminated. Each process maintains a set of variables needed for the Dijkstra-Scholten algorithm: an active flag to represent if the process is currently computing, a parent variable that stores the process responsible for the current dependency, a list of children and a child_count to keep track of processes that are waiting for an acknowledgement.

When the algorithm starts, each process initializes its variables and the initiator process is marked as active. During the execution, whenever a process sends a message to another process, it creates a dependency by adding the receiver as a child. When a process receives a message, if it does not have a parent yet, it assigns the sender as its parent. If the sender is not its parent, the process immediately sends an acknowledgement back, avoiding the creation of multiple parents.

The main component of the algorithm is the termination detection thread, which continuously waits for incoming control messages using MPI_Probe(). When an acknowledgement message is received, the process removes that child from its

list and checks if it can terminate. A process can only acknowledge its parent when it is passive and has no remaining children. This ensures that the termination information travels backwards through the dependency tree until it reaches the initiator. When the initiator receives all acknowledgements and becomes passive, it broadcasts the `TERMINATE_MESSAGE` to all processes.

The hooks are responsible for updating the termination detection state. When a process becomes active, the active variable is set to true. When it becomes passive, the process checks if it can finish by calling `try_finish()`. When sending a message, the destination process is added as a child, creating the dependency relationship. When receiving a message, the process either sets the sender as its parent or sends an acknowledgement if the sender is not its parent.

The biggest problem encountered during the implementation was maintaining the correct dependency tree. Since a process can receive multiple messages from the same process, simply increasing the child counter could lead to incorrect termination detection. To solve this, we created a boolean array to store which processes are already children, ensuring that each dependency is only counted once. Another issue was the possibility of race conditions between the computation threads and the termination detection thread, which was solved using a mutex to protect shared variables such as active, parent and `child_count`.

Safra

First we defined 2 messages, a `TERMINATE_MESSAGE` and a `TOKEN_MESSAGE`, the first for signaling termination and the second to notify when the token is arriving to the process. Next, we define that each process will hold a count (`proc_count`), a color (`proc_color`) and if it's currently active or not (active), we also assigned the id of the next process in the sequence in variable called `next`, as well as a `saved_token` and a `has_token` variables used for better management of the token. The token was defined as a struct containing a color and a count, in order to pass a struct in MPI messages we created a custom type `MPI_TOKEN_TYPE`, initialized in the `init_token_type()` function.

As soon as the algorithm starts we define the number of processes as well as its ids and next values. After that we initialize the token, with `count = 0` and `color = WHITE`, and tell the process with `id = 0` to start sending the token. The main component is a `while true` loop, to ensure that the processes are always listening for a message, after receiving one, they see if it's a token message or a termination message, in case of the second, they just terminate themselves, if the first happens, they update their `saved_token` and if the process id is 0, checks for termination. After all of this happens, the token is just updated and sent immediately if the process is passive, or held onto until it becomes passive, this update and transmission is done with the help of the `forward_token()`

function, which just makes the logic of the algorithm and sends the token to the next.

In this algorithms the hooks are actually very simple, when a process starts we just ensure it's values are as expected count = 0 and color = WHITE. when sending a message the count increases 1 and, if the id of the process who sent is bigger than the id of the process which is going to receive it , to color of the process changes to BLACK. When receiving a message the count just decreases one. In the activation hook the process just updates to true its active variable, in the passive hook, the opposite happens and if the has_token is true, the process sends the saved_token to the next to continue the algorithm.

The biggest problem we had was, how to terminate all of the processes, before we had the TERMINATE_MESSAGE we just closed the process that announces the termination, the process 0. After realizing that the other processes also has to terminate, we made a for loop, sending this TERMINATE_MESSAGE to all of the processes, and adapted their receive message methods to receive both token and terminate messages.

Compile and Run Recipes

To compile either of the algorithms, the user should only need to do make ALG=value, where value can either be safra or dijktra, a make clean before is also recommended but not necessary. To run the algorithms the user only needs to make mpirun -np n ./termination in the correct directory and switching the n by the desired number of processes.